1.

API Mocking using Mockito

Introduction & Basics

1. What is Mocking ?

- Mocking is a technique to create fake objects (mocks) that simulate the behavior of real dependencies.
- It allows us to test a class in isolation without relying on external systems like databases, APIs, or other services.
- Mocks return predefined responses instead of calling the actual dependency.

2. What is Mockito ?

- Mockito is a popular Java mocking framework used in unit testing.
- It helps to create, configure and verify mocks easily using simple syntax.
- It works with JUnit, TestNG and other testing frameworks.
- It is open-source and widely used in industry.

3. Why is mocking needed ?

- Avoids dependency on external systems (DB, APIs, Kafka, etc.).
- Faster and reliable test execution.
- Helps to test only the business logic.
- Allows testing of positive and negative scenarios easily.
- Reduces cost and maintenance effort.

4. Real vs Mocked Dependency

Aspect	Real Dependency	Mocked Dependency
Data	Actual data	Predefined fake data
Speed	Slower	Faster
Reliability	May be flaky	Stable and predictable
External Calls	Calls real system	No external calls
Use Case	Integration testing	Unit testing

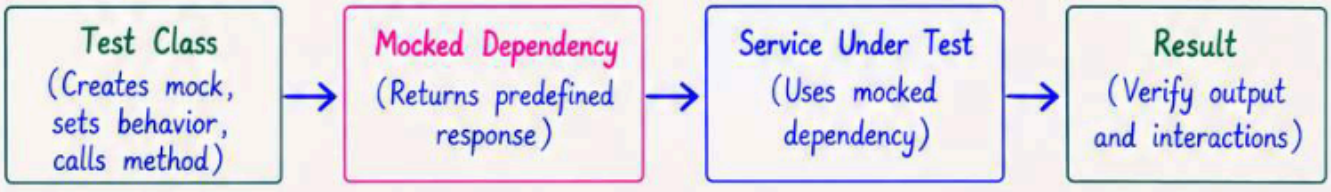
5. Where is it used ?

- Service layer unit tests.
- API client testing (RestTemplate, WebClient, Feign).
- Repository / DAO layer testing.
- Event-driven systems (Kafka).
- Utility and helper classes.
- Widely used in backend development and automation testing (SDET) roles.

6. Common things we can mock

- Repository / DAO
- External API clients
- Kafka Producer / Consumer
- Email / Notification services
- File system
- Static methods (with extensions)
- Database calls
- Web services
- Third-party SDKs
- Cache (Redis)
- Configuration services
- Any dependency class

7. Simple flow of mocking



8. Key takeaway

★ Mocking using Mockito helps us write fast, reliable and isolated unit tests by replacing real dependencies with controlled fake objects.

2. Mockito setup and first test

1. What dependencies are needed ?

- mockito-core : Core Mockito library for creating and using mocks.
- mockito-junit-jupiter : Integration with JUnit 5 (provides MockitoExtension).
- junit-jupiter : JUnit 5 testing framework (API and test engine).
- All should be added with test scope so they are only used in tests.

Maven dependencies (pom.xml)

```xml
<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.mockito</groupId>
    <artifactId>mockito-core</artifactId>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.mockito</groupId>
    <artifactId>mockito-junit-jupiter</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>
```

2. Two ways to initialize mocks

- Using MockitoExtension (recommended for JUnit 5)
  - Add @ExtendWith(MockitoExtension.class) to the test class.
- Using MockitoAnnotations.openMocks(this)
  - Call in a @BeforeEach method.
  - Useful when not using the extension.

Way 1: Using MockitoExtension (Recommended)

```java
@ExtendWith(MockitoExtension.class)
class PaymentServiceTest {
    @Mock
    private PaymentGateway gateway;
    @InjectMocks
    private PaymentService paymentService;
}
```

Way 2: Using openMocks(this)

```java
class PaymentServiceTest {
    @Mock
    private PaymentGateway gateway;
    @InjectMocks
    private PaymentService paymentService;

    @BeforeEach
    void setUp() {
        MockitoAnnotations.openMocks(this);
}   }
```

3. Basic unit test structure

- Class under test : The real class whose behavior we want to test (e.g., PaymentService).
- Dependency : A collaborator that will be mocked (e.g., PaymentGateway).
- Arrange : Set up mocks and input data.
- Act : Call the method on the class under test.
- Assert : Verify the result and interactions.

4. Real example: PaymentService

Service code (PaymentService.java)

```java
class PaymentService {
    private PaymentGateway gateway;

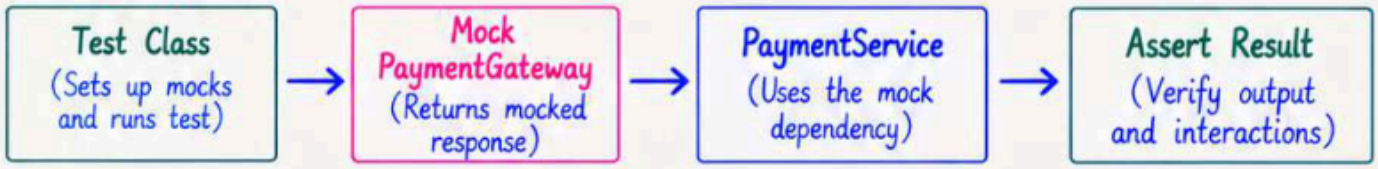
    public PaymentService(
        PaymentGateway gateway) {
        this.gateway = gateway;
    }

    public boolean processPayment(
        String orderId, double amount) {
        return gateway.charge(orderId,
                amount);
    }
}
```

Test code (PaymentServiceTest.java)

```java
@ExtendWith(MockitoExtension.class)
class PaymentServiceTest {
    @Mock
    private PaymentGateway gateway;
    @InjectMocks
    private PaymentService paymentService;
    @Test
    void testProcessPayment() {
        Mockito.when(gateway.charge(
          "ORD123", 100.0)).thenReturn(true);
        boolean result = paymentService.
          processPayment("ORD123", 100.0);
        assertTrue(result);
        Mockito.verify(gateway).charge(
    }     "ORD123", 100.0);
}
```

5. Simple flow of the test



6. Key takeaway

★ Mockito setup is simple and helps start isolated unit tests quickly.

3.

Core Mockito annotations

1. @Mock – Creates Fake Dependency

- @Mock creates a mock object (fake dependency) of a class or interface.
- It is used to define and configure the behavior of that dependency in tests.
- The mock does not call real methods.

2. @InjectMocks – Injects Mocks

- @InjectMocks creates an instance of the class under test and injects the mocks into it.
- Mockito injects mocks using constructor, setter, or field injection.
- Reduces boilerplate code in test setup.

3. @Spy – Partial Mock

- @Spy creates a partial mock of a real object.
- Calls real methods by default, unless specifically stubbed.
- Useful when you want to test real logic with some methods mocked.

4. @Captor – Captures Arguments

- @Captor creates an ArgumentCaptor to capture arguments passed to a mock.
- Helps in verifying the actual values passed during method calls.
- Useful when the argument is complex or needs detailed assertions.

5. Annotations Comparison

Annotation	Purpose	Example use
@Mock	Creates a mock object (fake dependency)	@Mock UserRepository repo;
@InjectMocks	Injects mocks into class under test	@InjectMocks OrderService service;
@Spy	Partial mock of a real object	@Spy List<String> list = new ArrayList<>();
@Captor	Captures arguments passed to a mock	@Captor ArgumentCaptor<Order> captor;

6. Example – OrderServiceTest

```
@ExtendWith(MockitoExtension.class)
class OrderServiceTest {
    @Mock
    private OrderRepository orderRepository;          // Mock for repository dependency
    @Mock
    private PaymentClient paymentClient;              // Mock for external client
    @InjectMocks
    private OrderService orderService;                // Class under test (mocks will be injected)
    @Captor
    private ArgumentCaptor<Order> orderCaptor;        // Captor to capture Order arguments
    // Test methods...
}
```

7. Simple Flow Using Annotations

Test Class (Add Mockito annotations) → **Mockito Processes Annotations** (Creates mocks, injects into class) → **Test Execution** (Use mocks, stubbing and verifications) → **Verify Results** (Assert behavior using mocks and captors)

8. Key Takeaway

★ Mockito annotations reduce boilerplate code and make unit tests clean, readable and maintainable by automatically creating mocks and injecting them into the class under test.

4.

Stubbing with Mockito

1. What is stubbing ?

- Stubbing means telling a mock what to return (or do) when a specific method is called.
- It helps simulate different scenarios in tests without calling real dependencies.

2. when(...).thenReturn(...)

- Most commonly used stubbing syntax.
- Specifies the value to return when a method is called with given arguments.

```
when(service.getName()).thenReturn("Mockito");
// service.getName() will return "Mockito"
```

3. Multiple Return Values

- Use thenReturn(a, b, ...) to return different values in consecutive calls.

```
when(service.getStatus())
     .thenReturn("UP", "DOWN", "UP");
// 1st call -> "UP"
// 2nd call -> "DOWN"
// 3rd call -> "UP"
```

4. thenThrow(...)

- Used to simulate exceptions.
- The specified exception is thrown when method is called.

```
when(service.fetchData()).thenThrow(
     new RuntimeException("Service failure"));
```

5. thenAnswer(...)

- Used for dynamic responses based on input arguments or invocation details.

```
when(service.getMessage(anyString()))
    .thenAnswer(invocation -> {
       String input = invocation.getArgument(0);
       return "Hello " + input;
    });
```

6. doReturn(...) / doThrow(...)

- Alternative stubbing style, mainly used with spies or for void methods.
- Helps avoid calling the real method (important for spies).

```
doReturn("Mocked").when(spyService).getName();

doThrow(new RuntimeException("Error"))
        .when(spyService).clearCache();
```

7. Example - InventoryClient & InventoryService

```
class InventoryClient {
    public int getStock(String itemId) {
        throw new RuntimeException(
             "Real service call");
    }
}

class InventoryService {
    private final InventoryClient client;

    public InventoryService(InventoryClient client) {
        this.client = client;
    }

    public int checkStock(String itemId) {
        return client.getStock(itemId);
    }
}
```

Success stub example

```
@Test
void testCheckStock_success() {
    InventoryClient client = mock(InventoryClient.class);
    when(client.getStock("A1")).thenReturn(100);

    InventoryService service = new InventoryService(client);
    int result = service.checkStock("A1");
    assertEquals(100, result);
}
```

Exception stub example

```
@Test
void testCheckStock_exception() {
    InventoryClient client = mock(InventoryClient.class);
    when(client.getStock("A1")).thenThrow(new RuntimeException("DB error"));

    InventoryService service = new InventoryService(client);
    assertThrows(RuntimeException.class, () ->
}                service.checkStock("A1"));
```

8. Common Stubbing Scenarios

Scenario	Stub	Expected Assertion
Return a single value	when(mock.method()).thenReturn(value)	assertEquals(value, ...)
Return multiple values	thenReturn(a, b, c)	assertEquals(a, ...), assertEquals(b, ...)
Throw an exception	thenThrow(new Exception(...))	assertThrows(Exception.class, ...)
Dynamic response	thenAnswer(...)	assert based on returned value
Use with spy / void	doReturn(...) / doThrow(...)	verify(...) or assert behavior

9. Key Takeaway

★ Stubbing defines how a mock should behave in each test.

5.

Verifying calls and interactions

1. Basic verification using verify()

- verify(mock).method() checks whether a method was called on the mock.
- It verifies the interaction, not the returned value.
- If the method was not called, the test will fail with a verification error.

```
verify(emailClient).sendEmail("user@test.com");
```

2. Verification modes

- times(n) : verifies method was called exactly n times.
- never() : verifies method was never called.
- atLeastOnce() : verifies method was called at least once.
- atMost(n) : verifies method was called at most n times.

3. No more interactions

- verifyNoMoreInteractions(mock) : ensures no other methods were called on the mock (apart from those already verified).
- verifyNoInteractions(mock) : ensures no methods were called on the mock at all.

4. InOrder verification

- InOrder allows verifying the order of method calls across one or more mocks.
- Useful when the sequence of interactions matters.

```
InOrder inOrder = inOrder(emailClient, smsClient);
inOrder.verify(emailClient).sendEmail(anyString());
inOrder.verify(smsClient).sendSms(anyString());
```

5. Common verification methods (quick reference)

Verification	Meaning	Example
verify(mock)	Verifies method was called (default 1 time)	verify(service).doWork();
times(n)	Verifies method was called exactly n times	verify(service, times(3)).doWork();
never()	Verifies method was never called	verify(service, never()).doWork();
atLeastOnce()	Verifies method was called at least once	verify(service, atLeastOnce()).doWork();
atMost(n)	Verifies method was called at most n times	verify(service, atMost(2)).doWork();
verifyNoMoreInteractions()	Ensures no other interactions happened	verifyNoMoreInteractions(service);
verifyNoInteractions()	Ensures no interactions at all	verifyNoInteractions(service);

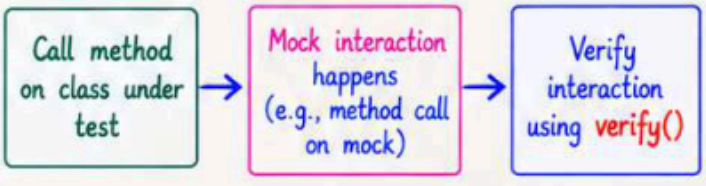
6. Code example

```
@Test
void shouldSendEmailAndNotSms() {
    NotificationService service = new NotificationService(
    emailClient, smsClient);

    service.sendNotification("user@test.com");

    verify(emailClient).sendEmail("user@test.com");
    verify(smsClient, never()).sendSms(anyString());
    verifyNoMoreInteractions(emailClient, smsClient);
}
```

7. Flow of verification



8. Key takeaway

★ Verification checks the behavior and interactions of your code with its dependencies, not just the returned data.

6.

Argument matchers and captors

1. Common matchers

- any() : Matches any value of any type.
- anyString() : Matches any String value.
- eq() : Matches an exact value.
- isNull() : Matches null.
- notNull(): Matches non-null value.

2. argThat() for custom checks

- argThat() allows custom matching logic using a predicate (lambda).
- Useful when built-in matchers are not enough.

Example :

```
when(service.save(argThat(req ->
        req.getId() > 0 && req.getName() != null)))
    .thenReturn(true);
```

3. Important note

- If matchers are used for one argument, use matchers for all applicable arguments in that method call, or use eq() for exact values.

Example (Correct) :

```
verify(service).sendEmail(any(), eq("SUBJECT"));
```

Example (Incorrect) :

```
verify(service).sendEmail(any(), "SUBJECT");
    // Mixing matcher and raw value
```

4. What is ArgumentCaptor ?

- ArgumentCaptor captures the actual arguments passed to a mocked method.
- Helps us verify the values and inspect fields of complex objects.
- Useful when we want to assert on multiple fields of the passed object.

5. Example: Capturing and asserting argument

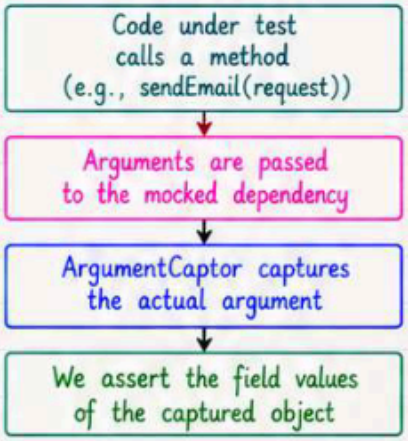
```
@Test
void testSendEmail() {
    ArgumentCaptor<EmailRequest> captor =
        ArgumentCaptor.forClass(EmailRequest.class);
    verify(emailService).sendEmail(captor.capture());

    EmailRequest request = captor.getValue();
    assertEquals("user@example.com", request.getTo());
    assertEquals("Welcome", request.getSubject());
    assertTrue(request.isHtml());
}
```

6. Matcher quick reference

Matcher	Use
any()	Matches any value of any type
anyString()	Matches any String value
eq(value)	Matches the exact value
isNull()	Matches null
notNull()	Matches non-null value
argThat()	Matches using custom condition (predicate)

7. Simple flow with ArgumentCaptor



8. Key takeaway

★ Argument matchers and captors help us validate how dependencies are called by matching or capturing the arguments passed to mock methods.

7.

Api mocking with RestTemplate

1. Why mock external API calls ?

- Avoids calling real external systems during unit tests.
- Tests are faster, reliable and not dependent on network, downtime or API changes.
- Helps to test only your service logic and response handling.
- Simulates different scenarios like success, error, timeout, etc.

2. Service example

- UserService calls an external User API using RestTemplate.
- It fetches user details from /users/{id} and returns a UserDto to the caller.
- In tests, we mock RestTemplate to return a fake response instead of making a real HTTP call.

3. UserService code (using RestTemplate)

```
@Service
public class UserService {
    private final RestTemplate restTemplate;

    public UserService(RestTemplate restTemplate) {
        this.restTemplate = restTemplate;
    }

    public UserDto getUserById(Long id) {
        String url = "https://api.example.com/users/" + id;
        ResponseEntity<UserDto> response =
            restTemplate.getForEntity(url, UserDto.class);
        return response.getBody();
    }
}
```

4. Test code (mocking RestTemplate)

```
@ExtendWith(MockitoExtension.class)
class UserServiceTest {
    @Mock
    private RestTemplate restTemplate;
    @InjectMocks
    private UserService userService;
    @Test
    void shouldReturnUser_WhenApiCallIsSuccessful() {
        Long userId = 1L;
        UserDto userDto = new UserDto(1L, "John", "john@example.com");
        ResponseEntity<UserDto> response =
                                ResponseEntity.ok(userDto);
        when(restTemplate.getForEntity(
                "https://api.example.com/users/" + userId,
                UserDto.class))
                .thenReturn(response);

        UserDto result = userService.getUserById(userId);

        assertNotNull(result);
        assertEquals("John", result.getName());
        assertEquals("john@example.com", result.getEmail());

        verify(restTemplate).getForEntity(anyString(), eq(UserDto.class));
    }
}
```

5. What to assert ?

- Response is not null (status handling).
- Data is correctly mapped to your DTO / model.
- Business logic works as expected (e.g., transformation, filtering, validation).
- Verify RestTemplate is called with the correct URL and parameters.

6. Request - Response flow (during test)

UserService (calls method) → RestTemplate Mock (returns fake response) → Fake API Response (ResponseEntity.ok (userDto)) → Assertion (verify data & interactions)

7. Real use cases

- Payment API - mock payment gateway responses.
- User Service - fetch user details from external system.
- Configuration Service - get feature flags or config values.

8. Key takeaway

★ Mock the API client (RestTemplate) to test your service logic without making real HTTP calls to external systems.

8.

Api response scenarios

1. Success response (200 / 201)

- Used when the API call is successful.
- 200 OK → typically for fetch/get operations.
- 201 Created → typically for create operations.
- Mock the API to return a successful ResponseEntity with expected data.

2. Not found (404)

- Used when the requested resource does not exist.
- Mock the API to return ResponseEntity with HttpStatus.NOT_FOUND or a custom error body.
- Verify that the service handles it gracefully (e.g., returns empty, throws custom exception, etc.).

3. Server error (500)

- Used when the server encounters an error.
- Mock the API to return HttpStatus.INTERNAL_SERVER_ERROR or throw HttpServerErrorException.
- Verify that the service handles the error (e.g., retries, throws exception, returns fallback).

4. Timeout or exception scenario

- Simulate network timeout or other exceptions like ResourceAccessException, SocketTimeoutException, or a generic RuntimeException.
- Verify that the service handles the exception properly (e.g., retry, fallback response, or meaningful error).

5. Comparison table

Scenario	Mockito stub	What to assert
Success (200)	thenReturn(ResponseEntity.ok(data))	Response is successful and contains expected data
Created (201)	thenReturn(new ResponseEntity<>(data, HttpStatus.CREATED))	Status is 201 and data is as expected
Not found (404)	thenReturn(ResponseEntity.status(HttpStatus.NOT_FOUND).build())	Service handles 404 (empty / custom exception)
Server error (500)	thenThrow(new HttpServerErrorException(HttpStatus.INTERNAL_SERVER_ERROR))	Service handles exception / fallback response
Timeout / Exception	thenThrow(new RuntimeException("Timeout"))	Service handles exception (retry / fallback / error)

6. Code snippets

Success response (200 OK)

```
UserResponse mockResponse = new UserResponse(1L, "Aman");
when(restTemplate.getForEntity(anyString(),
        eq(UserResponse.class)))
    .thenReturn(ResponseEntity.ok(mockResponse));
```

Server error / Exception

```
when(restTemplate.getForEntity(anyString(1;
                    eq(UserResponse.class)))
    .thenThrow(new RuntimeException("API failed"));
// or specifically HTTP 500
    .thenThrow(new HttpServerErrorException(
                    HttpStatus.INTERNAL_SERVER_ERROR));
```

7. Positive vs Negative test cases

- Positive test cases: Verify expected behavior for successful responses (200 / 201).
- Negative test cases: Verify behavior for failure responses (404 / 500) and exceptions (timeout, etc.).

8. Simple flow of testing API response

Stub Scenario (200 / 404 / 500 / Exception) → Call Service Method (which uses mocked API) → Assert Result / Exception / Fallback

9. Key takeaway

★ Good tests cover both happy path and failure responses, ensuring the service is reliable, resilient and handles real-world scenarios.

9.

Kafka mocking for producers

1. Why mock Kafka in unit tests ?

- Avoids dependency on a real Kafka broker.
- Faster and more reliable unit tests.
- Focuses on the business logic (what event is sent) instead of infrastructure.
- Allows verification of topic, event data, and headers if needed.
- No need for external setup (Kafka, Docker, etc.).

2. Producer service example

- A typical Spring Boot service uses KafkaTemplate to publish events to Kafka.
- We can use KafkaTemplate<String, String> for simple string events.
- Or KafkaTemplate<String, OrderEvent> for structured events.
- In this example, we'll use KafkaTemplate<String, OrderEvent> and a simple publisher service.

3. Producer service code (publish method)

OrderEventPublisher.java

```
@Service
public class OrderEventPublisher {
    private final KafkaTemplate<String, OrderEvent>
        kafkaTemplate;

    public OrderEventPublisher(
        KafkaTemplate<String, OrderEvent> kafkaTemplate) {
        this.kafkaTemplate = kafkaTemplate;
    }

    public void publishOrderCreated(OrderEvent event) {
        String topic = "order-events";
        kafkaTemplate.send(topic, event);
    }
}
```

4. Unit test code (mocking KafkaTemplate)

OrderEventPublisherTest.java

```
@ExtendWith(MockitoExtension.class)
class OrderEventPublisherTest {
    @Mock
    private KafkaTemplate<String, OrderEvent>
        kafkaTemplate;

    @InjectMocks
    private OrderEventPublisher publisher;

    @Test
    void shouldPublishOrderCreatedEvent() {
        OrderEvent event = new OrderEvent("123", "CREATED");
        publisher.publishOrderCreated(event);

        verify(kafkaTemplate).send("order-events", event);
    }
}
```

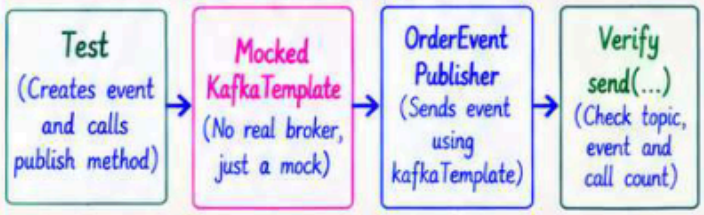
5. Verification examples

```
// Verify send was called with exact topic and event
verify(kafkaTemplate).send("order-events", event);

// Verify send was called exactly once
verify(kafkaTemplate, times(1)).send("order-events", event);

// Verify send was called with any event
verify(kafkaTemplate, times(1)).send(eq("order-events"),
    any(OrderEvent.class));
```

6. Simple flow of mocking



7. Real use cases

- Order events - publish order created, updated, cancelled events.
- Notifications - publish user notification events.
- Audit events - publish audit logs to Kafka.

8. Key takeaway

★ Mock the KafkaTemplate to validate your event publishing logic in unit tests without requiring a real Kafka broker. Verify that the correct topic and event are sent, and use Mockito verify() to assert interactions.

10.

Kafka consumer test example

1. Consumer / Listener role

- A Kafka consumer (listener) reads messages from a Kafka topic in an event-driven system.
- It processes the event and performs business logic, such as calling another service.
- In tests, we focus on the listener logic by mocking downstream dependencies, not the real Kafka broker.

2. Example component

- OrderEventListener listens to order events from a Kafka topic.
- When an event is received, it calls OrderService to process the order.
- It may also call NotificationService to send a notification (e.g., order confirmation).

3. Listener code (simple example)

```
@Component
public class OrderEventListener {
    private final OrderService orderService;
    private final NotificationService notificationService;

    public void onMessage(OrderEvent event) {
        orderService.process(event);
        notificationService.sendOrderConfirmation(event);
    }
}
```

4. Test code (mocking downstream service)

```
@ExtendWith(MockitoExtension.class)
class OrderEventListenerTest {
    @Mock
    private OrderService orderService;
    @Mock
    private NotificationService notificationService;
    @InjectMocks
    private OrderEventListener listener;

    @Test
    void shouldProcessOrderEvent() {
        OrderEvent event = new OrderEvent("123", "NEW");
        listener.onMessage(event);

        verify(orderService).process(event);
        verify(notificationService).sendOrderConfirmation(event);
    }
}
```

5. Test scenarios

- Success scenario: Event is processed and both services are called.
- Exception scenario: orderService throws an exception and notificationService is not called.

6. Example exception test

```
@Test
void shouldHandleServiceException() {
    OrderEvent event = new OrderEvent("123", "NEW");
    doThrow(new RuntimeException("DB error"))
            .when(orderService).process(event);

    assertThrows(RuntimeException.class, () ->
        listener.onMessage(event));
    verify(orderService).process(event);
    verify(notificationService, never()).sendOrderConfirmation(any());
}
```

7. Scenario summary

Scenario	Mock behavior	Expected verification
Success	No exception	verify(orderService).process(event) verify(notificationService).sendOrderConfirmation(event)
Exception	orderService throws RuntimeException	verify(orderService).process(event) verify(notificationService, never()).sendOrderConfirmation(any())

8. Simple flow of testing Kafka consumer

Kafka Message (OrderEvent) → OrderEventListener (onMessage) → Mocked Services (OrderService, NotificationService) → Verify Processing (Mockito verify)

9. Key takeaway

★ Test Kafka consumer (listener) behavior by mocking downstream dependencies, not the real Kafka broker.

11.

Spy And Partial Mocking

@ Aman_Views

1. What is a spy ?

- A spy is a real object wrapped by Mockito that allows you to stub specific methods while keeping the real behavior for other methods.
- It is also called partial mocking because only certain methods are mocked and the rest call the real implementation.
- Useful when you want to test a class but replace only some of its behavior.

2. Mock vs Spy Comparison

Feature	Mock (@Mock)	Spy (@Spy)
Object type	Mock object (created by Mockito)	Real object (wrapped by Mockito)
Method behavior	All methods return default values (unless stubbed)	Calls real methods (unless specifically stubbed)
Use case	Isolate and mock dependencies	Partial mocking of a real object
Creation	@Mock	@Spy or Mockito.spy(object)
Stubbing	when(...).thenReturn(...)	doReturn(...).when(...)
Real method call	No (by default)	Yes (unless stubbed)

3. When to use @Spy ?

- When you have a complex class and want to mock only specific methods.
- When calling the real methods is desired for most of the behavior.
- When the class has non-mockable behavior or internal state that should be used.
- Useful in service or utility classes where only a few external calls (e.g., API, DB) need to be stubbed.

4. Example - Real service with one method stubbed

- Consider a PaymentService that calls an external API to process payment and has other helper methods.
- We will spy the real service and stub only the call to the external API (charge()).
- Other methods like generateReceipt() will execute the real implementation.
- This allows us to test the service logic without making an actual external API call.

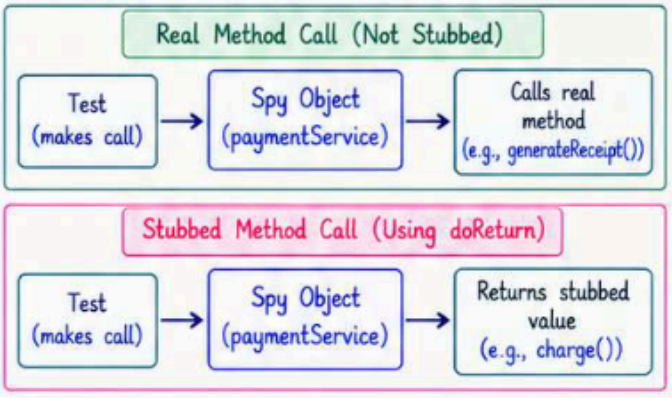
5. Code example using @Spy and doReturn(...)

```
@ExtendWith(MockitoExtension.class)
class PaymentServiceTest {
    @Spy
    private PaymentService paymentService;

    @Test
    void testProcessPayment_withStubbedCharge() {
        // Stub only the charge() method
        doReturn(true).when(paymentService).charge("ORD123", 100.0);

        // Call the real method (uses stubbed charge())
        boolean result = paymentService.processPayment("ORD123", 100.0);
        assertTrue(result);
    }
}
```

6. Real vs Stubbed Method Flow



7. Key takeaway

★ Use @Spy for partial mocking when you want to stub specific methods and keep the real behavior for others. It is helpful in testing real service classes while avoiding external calls like APIs, databases, etc.

12.

Handling Exceptions And Void Methods

@Aman_Views

1. Mocking failures with thenThrow

- Use thenThrow() to make a mock throw an exception when a method is called.
- Helpful to test error handling logic in your code.
- You can throw runtime or checked exceptions.

Example: thenThrow()

```
@Test
void shouldThrowExceptionWhenUserNotFound() {
    when(userService.getUserById(1L))
        .thenThrow(new RuntimeException("User not found"));

    assertThrows(RuntimeException.class, () -> {
        userService.getUserById(1L);
    });
}
```

2. Mocking void methods with doNothing and doThrow

- Void methods do not return a value, so we use doNothing(), doThrow(), doAnswer() etc.
- doNothing() is the default behavior (no action).
- Use doThrow() to simulate failures in void methods.

Examples: Void methods

```
@Test
void shouldDoNothingOnValidInput() {
    doNothing().when(notificationService).sendEmail(anyString());
    notificationService.sendEmail("test@example.com");
    verify(notificationService).sendEmail("test@example.com");
}
@Test
void shouldThrowExceptionOnEmailFailure() {
    doThrow(new RuntimeException("Email service down"))
        .when(notificationService).sendEmail(anyString());

    assertThrows(RuntimeException.class, () -> {
        notificationService.sendEmail("test@example.com");
    });
}
```

3. Sequential stubbing for retry scenarios

- Use thenReturn() and thenThrow() in sequence to simulate different responses on multiple calls.
- Useful for testing retry logic (e.g., first call fails, second call succeeds).

Example: Sequential stubbing

```
@Test
void shouldSucceedOnRetry() {
    when(paymentService.processPayment(any()))
        .thenThrow(new RuntimeException("Temporary error"))
        .thenReturn("PAYMENT_SUCCESS");

    String result = paymentService.processPayment("order123");
    assertEquals("PAYMENT_SUCCESS", result);
    verify(paymentService, times(2)).processPayment("order123");
}
```

4. API error response example

- Mock external API client to throw exceptions like HTTP 500 or timeout.
- Helps test how your service handles API failures.

Example: API client throwing error

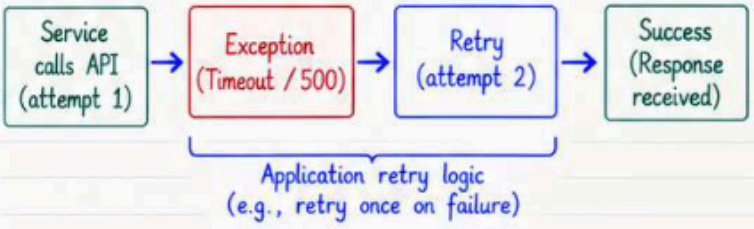
```
@Test
void shouldHandleApiError() {
    when(apiClient.callExternalApi())
        .thenThrow(new RuntimeException("HTTP 500 - Internal Error"));

    assertThrows(RuntimeException.class, () -> {
        orderService.syncOrder();
    });
    verify(apiClient).callExternalApi();
}
```

5. Common Mockito methods for exceptions and void

Method	Description	Use Case
thenThrow()	Throw exception for a method	Simulate failures
doNothing()	Do nothing for void method	Default behavior
doThrow()	Throw exception for void method	Simulate void failures
thenReturn()	Return value (can be chained)	Sequential stubbing
thenAnswer()	Custom logic/response	Advanced scenarios

6. Retry flow example (first call fails, second succeeds)



7. Key takeaway

★ Use thenThrow(), doThrow() and sequential stubbing to test error handling, void methods and retry logic effectively.

13.

ArgumentCaptor And InOrder

@Aman_Views

1. What is ArgumentCaptor ?

- ArgumentCaptor is a Mockito utility to capture arguments passed to a mock method.
- Useful when we want to verify the actual values of complex objects (e.g., DTO, event) sent to a dependency.
- We capture the argument and make assertions on its fields.
- Commonly used for request bodies, events, callbacks, etc.

2. Capturing request DTO or event payload

- Create an ArgumentCaptor for the required type.
- Use captor.capture() in the verify() call.
- Get the captured value using getValue() or getAllValues().
- Assert the fields of the captured object.
- Works well for REST requests, Kafka events, messages, etc.

3. Verifying call sequence with InOrder

- InOrder allows us to verify that multiple interactions happen in a specific order.
- Create an InOrder instance with one or more mocks.
- Use inOrder.verify(mock).method() in the expected sequence.
- Useful when the order of method calls matters (e.g., save → publish → notify).

4. Small verify(), times(), never() summary

Method	Meaning	Example
verify(mock)	Verify method was called (once by default).	verify(service).process(req);
times(n)	Verify method was called n times.	verify(service, times(3)).process();
never()	Verify method was never called.	verify(service, never()).process();

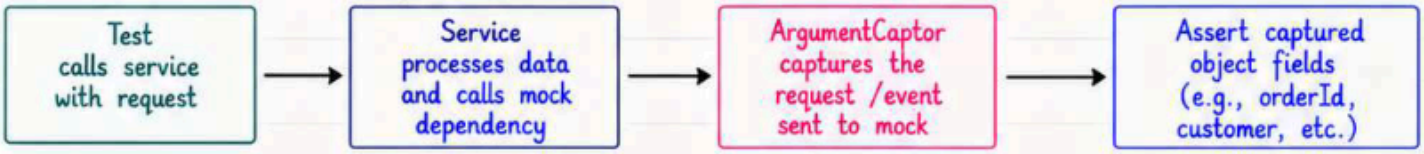
5. Java code example capturing an API request body or event

```
@ExtendWith(MockitoExtension.class)
class OrderServiceTest {
    @Mock   private OrderClient orderClient;
    @InjectMocks   private OrderService orderService;
    @Test
    void shouldSendOrderRequestWithCorrectData() {
        // Act
        OrderRequest req = new OrderRequest("ORD123", "John", 2);
        orderService.placeOrder(req);

        // Capture the argument passed to the mock
        ArgumentCaptor<OrderRequest> captor =
              ArgumentCaptor.forClass(OrderRequest.class);
        verify(orderClient).sendOrder(captor.capture());

        // Assert on captured request
        OrderRequest captured = captor.getValue();
        assertEquals( "ORD123", captured.getOrderId());
        assertEquals( "John", captured.getCustomerName());
        assertEquals(2, captured.getQuantity());
    }
}
```

6. Mini interaction flow diagram



7. Key takeaway

★ Use ArgumentCaptor to verify the actual data passed to mocks and InOrder to ensure method calls happen in the correct order.

14.

Mockito With Spring Boot

@Aman_Views

1. Where Mockito fits in Spring Boot tests ?

- Mockito is used to create mocks for dependencies in Spring Boot tests.
- Helps to isolate the component being tested (controller, service, etc.).
- Used with JUnit 5 and Spring Test (spring-boot-starter-test).
- Useful for unit tests (fast) and slice tests (e.g., @WebMvcTest, @DataJpaTest).

2. @MockBean vs @Mock

Feature	@Mock	@MockBean
Provided by	Mockito	Spring Boot Test
Used in	Unit tests (plain Mockito)	Spring Boot tests (with Spring context)
Registers in Spring Context	No	Yes (replaces real bean)
Scope	Only inside test class	In Spring ApplicationContext
Typical use case	Service unit tests (with @InjectMocks)	Controller / Slice / Integration tests
Example	@Mock private UserRepository repo;	@MockBean private UserRepository repo;

3. Service-layer API example (mocking repository)

```
@Service
public class UserService {
    private final UserRepository userRepository;

    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public UserDto getUserById(Long id) {
        return userRepository.findById(id)
                .map(user -> new UserDto(user.getId(), user.getName()))
                .orElseThrow(() -> new RuntimeException("User not found"));
    }
}
```

4. Spring Boot test example (using @MockBean)

```
@SpringBootTest
class UserServiceTest {
    @Autowired
    private UserService userService;

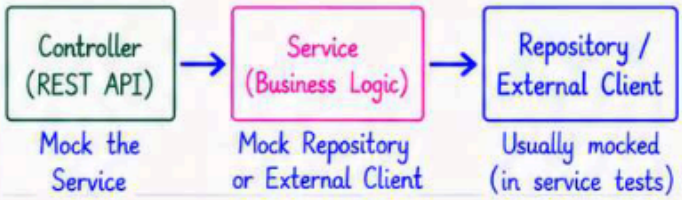
    @MockBean
    private UserRepository userRepository;

    @Test
    void getUserById_shouldReturnUser() {
        User user = new User(1L, "John");
        when(userRepository.findById(1L)).thenReturn(Optional.of(user));
        UserDto result = userService.getUserById(1L);
        assertNotNull(result);
        assertEquals("John", result.getName());
        verify(userRepository).findById(1L);
    }
}
```

5. What to mock in different layers ?

Layer	What to mock	Examples	Notes
Controller	Service layer	@MockBean UserService	Test only controller logic (request/response).
Service	Repository or External client	@Mock UserRepository @Mock RestTemplate	Test business logic. Do not hit real DB or external APIs.
Repository	Usually nothing	Use in-memory DB (e.g., H2) if needed	Test with @DataJpaTest (Not usually mocked).

6. Simple architecture (what we test)



7. Key takeaway

- Use @MockBean in Spring Boot tests to replace real beans in the application context.
- Mock only external dependencies and test one layer at a time (controller, service, etc.) to keep tests fast and reliable.

15.

Best Practices And Common Mistakes

@Aman_Views

1. Mockito best practices

- Write focused unit tests (test one behavior at a time).
- Use meaningful test and method names.
- Prefer MockitoExtension (JUnit 5) for easy setup.
- Use @Mock and @InjectMocks annotations.
- Mock only external dependencies, not your own logic.
- Use verify() to check important interactions.
- Use argument matchers (any(), eq()) when needed.
- Keep tests simple, readable and maintainable.
- Clean up unnecessary stubbing (use lenient() if required).

2. Common mistakes to avoid

- Mocking your own class (tests become meaningless).
- Over-mocking (mocking too many objects).
- Not using verify() (only checking return values).
- Using when() for void methods (should use doNothing(), doThrow(), etc.).
- Ignoring argument matchers (mixing raw values and matchers).
- Writing complex test logic (tests should be simple).
- Not testing important edge cases (exceptions, empty data).
- Relying too much on mocks instead of real behavior.
- Verifying implementation details instead of behavior.

3. What should be mocked and what should not be mocked

Mock This (External Dependencies)	Do Not Mock This
External services (e.g., payment API)	Your class under test
Repositories / DAOs (e.g., database)	Simple POJOs / DTOs
HTTP clients (e.g., RestTemplate, WebClient)	Value objects and models
Third-party libraries (e.g., email, SMS, cloud)	Utility classes (unless they call external systems)
Other microservices / clients	Business logic that you want to actually test

4. Short checklist for readable test cases

- ☐ Test name clearly describes the scenario.
- ☐ Mocks are set up correctly.
- ☐ Arrange - Act - Assert structure is followed.
- ☐ Only necessary dependencies are mocked.
- ☐ Important interactions are verified using verify().
- ☐ Edge cases (success, failure, empty, exception) are covered.
- ☐ No unnecessary stubbing or complex logic.
- ☐ Test is easy to understand and maintain.
- ☐ Uses meaningful data (not just random values).

5. Example: Clear Arrange - Act - Assert structure

```
@ExtendWith(MockitoExtension.class)
class OrderServiceTest {
    @Mock
    private PaymentGateway paymentGateway;
    @InjectMocks
    private OrderService orderService;

    @Test
    void shouldPlaceOrderSuccessfully() {
        // Arrange
        when(paymentGateway.charge("ORD123", 100.0)).thenReturn(true);

        // Act
        boolean result = orderService.placeOrder("ORD123", 100.0);

        // Assert
        assertTrue(result);
        verify(paymentGateway).charge("ORD123", 100.0);
    }
}
```

6. Summary table: Do vs Avoid

Topic	Do (Recommended)	Avoid (Not Recommended)
Test Scope	Test one behavior at a time	Testing multiple behaviors in one test
Mocking	Mock external dependencies	Mocking your own class
Annotations	Use @Mock, @InjectMocks and MockitoExtension	Manual and complex setup
Stubbing	Use when() and doReturn() appropriately	Using when() for void methods
Verification	Use verify() for important interactions	Skipping verification
Test Design	Keep tests simple and readable	Complex, hard to understand tests
Data	Use meaningful test data	Using random or unclear data

7. Final key takeaway

★ Use Mockito wisely – mock only what you don't control, keep tests simple, verify important behavior, and avoid common pitfalls to write reliable and maintainable unit tests.